

# Dag

## Project Structure

This section provides an overview of Project Structure.

The source code of Dags mainly includes:

- **Node**: Abstraction of a task.
- **Graph**: A network that runs `Node`s asynchronously.
- **Connection**: Interfaces for communication between `Node`s.
- **Parser**: Help simplify `Node` creations and analyze connections from user input.

## Node

Abstraction of a task.

In Dags, we provide the trait `Node` as the representation of user's task. `Node` is the basic scheduling unit of [Graph](#).

## Node's interfaces

```
///# The ['Node'] trait
///
/// Nodes are the basic scheduling units of Graph. They can be identified by
/// a globally assigned ['NodeId'] and a user-provided name.
///
/// Nodes can communicate with others asynchronously through ['InChannels']
/// and ['OutChannels'].
///
/// In addition to the above properties, users can also customize some other
/// attributes.
#[async_trait]
pub trait Node: Send + Sync {
    /// id is the unique identifier of each node, it will be assigned by the
    /// ['NodeTable']
    /// when creating a new node, you can find this node through this
    /// identifier.
    fn id(&self) -> NodeId;
    /// The node's name.
    fn name(&self) -> NodeName;
```

```

    /// Input Channels of this node.
    fn input_channels(&mut self) -> &mut InChannels;
    /// Output Channels of this node.
    fn output_channels(&mut self) -> &mut OutChannels;
    /// Execute a run of this node.
    async fn run(&mut self, env: Arc<EnvVar>) -> Output;
    /// Return true if this node is conditional node. By default, it returns
false.
    fn is_condition(&self) -> bool {
        false
    }
    /// Returns the list of nodes that are part of this node's loop structure,
if any.
    ///
    /// This method is used to identify nodes that are part of a loop-like
structure, such as a loop subgraph.
    /// When this method returns Some(nodes), the loop detection check will
skip checking these nodes for cycles.
    ///
    /// Returns None by default, indicating this is not a loop-containing
node.
    fn loop_structure(&self) -> Option<Vec<Arc<Mutex<dyn Node>>>> {
        None
    }

    /// Returns true if this node has TypedContent input.
    /// By default, it returns false.
    fn has_typed_input(&self) -> bool {
        false
    }

    /// Returns true if this node has TypedContent output.
    /// By default, it returns false.
    fn has_typed_output(&self) -> bool {
        false
    }
}

```

The above methods ensure the communication and execution function when `Graph` schedules Tasks.

## Dynamic execution logics

It is possible to execute different logic on the same data layout. So we provide trait `Action` as the abstraction of a task's logic.

Here is an example of implementing `Action`, that simply returns `Output::Out` containing a `String "Hello world"`.

```
use dagra::{Action, InChannels, OutChannels, EnvVar, Output};
/// An implementation of ['Action'] that returns ['Output::Out'] containing a
String "Hello world".
#[derive(Default)]
pub struct HelloAction;
#[async_trait]
impl Action for HelloAction {
    async fn run(&self, _: &mut InChannels, _: &OutChannels, _: Arc<EnvVar>) -
> Output {
        Output::Out(Some(Content::new("Hello world".to_string())))
    }
}
```

## Create a node

### Get a node id

Before creating a new node, let's first get to know the `NodeTable` - a mapping of nodes' names to their ids. During runtime, we need to figure out the specific node to send/receive messages to/from it. Therefore, each node will be assigned a unique id before the run.

You can call the `alloc_id_for` method of `NodeTable` to alloc an id by providing a name, just like the following example.

```
use dagra::NodeTable;
let node_table = NodeTable::default();
let name = "Node Name";
let id = node_table.alloc_id_for(name);
```

### Create a default node

If you don't need a `Node` to hold any extra information, `DefaultNode` is the best choice.

We can create a `DefaultNode` with a custom `Action` and other necceray fields. The example below creates a `DefaultNode` with a name of "My Node", an action of `HelloAction` (defined above), a `NodeId` assigned by the `node_table`, and empty `input_channels` and `output_channels`.

```
use dagra::{NodeName, NodeTable, DefaultNode, EmptyAction};
```

```

let node_name = "My Node";
let mut node_table = NodeTable::new();
let mut node = DefaultNode::with_action(
    NodeName::from(node_name),
    HelloAction::default(),
    &mut node_table,
);

// Check if node table has key-value pair (node.name, node.id)
assert_eq!(node_table.get(node_name).unwrap(), &node.id());

let env = Arc::new(EnvVar::new(node_table));
let out = node.run(env).get_out().unwrap();
let out: &String = out.get().unwrap();
assert_eq!(out, "Hello world");

```

## Implement a customized node

Don't worry if the `DefaultNode` can't satisfy your need. You are free to implement a customized `Node` like the following example:

```

use std::sync::Arc;
use async_trait::async_trait;
use dagrs::{Content, EnvVar, InChannels, Node, NodeId, NodeName, OutChannels, Output};

struct MessageNode {
    id: NodeId,
    name: NodeName,
    in_channels: InChannels,
    out_channels: OutChannels,
    /*Put your custom fields here.*/
    message: String,
}

#[async_trait]
impl Node for MessageNode {
    fn id(&self) -> NodeId {
        self.id
    }

    fn name(&self) -> NodeName {
        self.name.clone()
    }
}

```

```

fn input_channels(&mut self) -> &mut InChannels {
    &mut self.in_channels
}

fn output_channels(&mut self) -> &mut OutChannels {
    &mut self.out_channels
}

async fn run(&mut self, _: Arc<EnvVar>) -> Output {
    Output::Out(Some(Content::new(self.message.clone())))
}
}

```

The `MessageNode` above simply returns its field `message` when run. If needed, the execution logic of this node can also be replaced with an `Action`, just like the `DefaultNode`.

If you need many custom Nodes like this, it can be annoying to duplicate these necceray fields and methods required by the `Node` trait. The [auto\\_node](#) macro can help you out.

## Connection

Let the nodes communicate asynchronously.

Since these nodes run on electronic devices, we need to transmit data beyond horseback.

Dagrs uses [tokio channels](#) to establish connections between nodes. You don't need to know how to create and use tokio channels, because Dagrs has encapsulated this mess and [Graph](#) will help you create and manage these channels.

## Information packet

We call the data passed between nodes "information packet", just like in Flow-based Programming. Dagrs provides type `Content` as the information packet.

## Create an information packet

If you want to send something like a greeting in the following example, first create an information packet.

```

use dagrs::Content

let greeting = Content::new("Hey there~").

```

How `Content` keeps data of different type:

The secret to sending information packets of different types on the same channel is the `Any` trait. When some data is encapsulated into a `Content`, they are converted into an `Arc` pointer.

## Unwrap an information packet

When you need to get the contents of the received packet, use the `get` method:

```
use dagrs::Content

let greeting = Content::new("Hey there~").
let recv_greeting: String = greeting.get().unwrap();
assert_eq!(&recv_greeting, "Hey there~");
```

Note:

The `get` method converts the `Content`'s `Arc` pointer to the type you annotated, or the type inferred by the compiler. If the conversion is successful, you will get `Some(T)`, otherwise you will get `None`.

## Send & Receive information packet

### Send a information packet

In Dagrs, each node maintains an `OutChannels` to manage channels from that node to other nodes. By providing the destination node id and the packet information, `OutChannels` will send the packet to the node you want.

Suppose we have two nodes: `a` and `b`, and we want node `a` to send a greeting to `b`. Then we need to put the greeting into the channel from `a` to `b` as the following example, which implements node `a`'s action - `GreetingAction`.

```
use dagrs::{Action, InChannels, OutChannels, EnvVar, Output};

const NAME_OF_B: &str = "b";

pub struct GreetingAction;
#[async_trait]
impl Action for GreetingAction {
    async fn run(&self, _: &mut InChannels, out_channel: &OutChannels, env: Arc<EnvVar>) -> Output {
        let greeting = Content::new("Hey there~").
        let b_id = env.get_node_id(NAME_OF_B).unwrap();
        out_channel.send_to(&b_id, greeting).unwrap();
    }
}
```

```

        Output::Out(None)
    }
}

```

The `send_to` method is asynchronous. You can also use the method `blocking_send`, which is synchronous.

## Receive a information packet

Each node maintains an `InChannels` to manage channels from other nodes to that node. By providing the source node id, `InChannels` will try to receive a packet from the node you want.

Following the same pattern above, the following example shows how node `b` receives a greeting from node `a` via the implementation of `RecvGreetingAction`.

```

use dagrs::{Action, InChannels, OutChannels, EnvVar, Output};

const NAME_OF_A: &str = "a";

pub struct RecvGreetingAction;
#[async_trait]
impl Action for RecvGreetingAction {
    async fn run(&self, in_channel: &mut InChannels, _: &OutChannels, env:
Arc<EnvVar>) -> Output {
        let a_id = env.get_node_id(NAME_OF_A).unwrap();
        let greeting = in_channel.recv_from(&a_id).unwrap().

        Output::Out(None)
    }
}

```

The `recv_from` method is asynchronous. You can also use the method `blocking_recv_from`, which is synchronous.

## Lifetime of a channel

Channels are created when the graph is constructed and the corresponding senders and receivers of these channels are assigned to nodes. Since all good things must come to an end, you can close senders/receivers manually, otherwise they will be closed automatically when the graph is deleted.

## Closing a sender

Let's continue with the above example, where node `a` closes the channel's sender after sending the greeting:

```
const NAME_OF_B: &str = "b";
#[async_trait]
impl Node for MyNode {
    // Other method implementations...
    async fn run(&mut self, env: Arc<EnvVar>) -> Output {
        // `action` sends the greeting to other nodes
        let output = self.action.run(&mut self.input_channels(),
&self.output_channels(), env.clone());
        let b_id = env.get_node_id(NAME_OF_B).unwrap();
        self.output_channels().close(&b_id);
        output
    }
}
```

After `a` closes the sender, the key-value pair of (`b`'s id, the closed sender) is removed from `a.out_channels`.

## Closing a receiver

When node `b` receives the greeting, it also closes the receiver:

```
use dagrs::{Action, InChannels, OutChannels, EnvVar, Output};

const NAME_OF_A: &str = "a";

pub struct RecvGreetingAction;
#[async_trait]
impl Action for RecvGreetingAction {
    async fn run(&self, in_channel: &mut InChannels, _: &OutChannels, env:
Arc<EnvVar>) -> Output {
        let a_id = env.get_node_id(NAME_OF_A).unwrap();
        let greeting = in_channel.recv_from(&a_id).unwrap().

        in_channel.close(&a_id);
        Output::Out(None)
    }
}
```

## Exception cases in send/receive

Things could go wrong even though they appear perfect.



## Error types of send

If a send operation fails, you will get an enum `SendErr` :

| Enum Variant of <code>SendErr</code> | Description                                                                              |
|--------------------------------------|------------------------------------------------------------------------------------------|
| <code>NoSuchChannel</code>           | No sender corresponding to the given node id.                                            |
| <code>ClosedChannel(Content)</code>  | The corresponding channel is closed already. Returns the content that failed to be sent. |

## Error types of receive

If a receive operation fails, you will get an enum `RecvErr` :

| Enum Variant of <code>RecvErr</code> | Description                                                                                                         |
|--------------------------------------|---------------------------------------------------------------------------------------------------------------------|
| <code>NoSuchChannel</code>           | No sender corresponding to the given node id.                                                                       |
| <code>Closed</code>                  | The corresponding channel is closed already.                                                                        |
| <code>Lagged(u64)</code>             | The channel experienced a buffer overflow, causing the receiver to discard a certain number of information packets. |

## Graph

Record the dependency relationships and operational results of nodes in the system

`Graph` is `dagrs`'s main body. `Graph` is a network that satisfies FBP logic, provides node dependencies, and runs all of its nodes completely asynchronously.

## Graph's structure

```
pub struct Graph {  
    /// Define the Net struct that holds all nodes  
    pub(crate) nodes: HashMap<NodeId, Arc<Mutex<dyn Node>>>,  
    /// Store a task's running result. Execution results will be read  
    /// and written asynchronously by several threads.  
    pub(crate) execute_states: HashMap<NodeId, Arc<ExecState>>,  
    /// Count all the nodes  
    pub(crate) node_count: usize,  
    /// Global environment variables for this Net job.  
    /// It should be set before the Net job runs.
```

```

pub(crate) env: Arc<EnvVar>,
/// Mark whether the net task can continue to execute.
/// When an error occurs during the execution of any task, This flag will
still be set to true
pub(crate) is_active: Arc<AtomicBool>,
/// Node's in_degree, used for check loop
pub(crate) in_degree: HashMap<NodeId, usize>,
/// Stores the blocks of nodes divided by conditional nodes.
/// Each block is a HashSet of NodeIds that represents a group of nodes
that will be executed together.
pub(crate) blocks: Vec<HashSet<NodeId>>,
/// Abstract representation of the graph structure, used for cycle
detection
pub(crate) abstract_graph: AbstractGraph,
}

```

The above fields provide information such as the storage of node operation results, node information contained in the graph, and graph operation status.

## Create a Graph

A `Graph` contains multiple nodes, which can be added as long as they implement the `[ Node ]` trait.

### add node

Firstly, it is necessary to initialize a node that implements the `Node` trait. Let's use `DefaultNode` as an example.

```

use dagra::NodeTable;
let mut node_table = NodeTable::new();
let node_name = "Node X";
let node = DefaultNode::new(NodeName::from(node_name), &mut node_table);
let node1_name = "Node Y";
let node1 = DefaultNode::with_action(
    NodeName::from(node1_name),
    HelloAction::new(),
    &mut node_table,
);

```

After calling the new function to create a default `Graph`, it is necessary to add nodes to the `Graph`, `Graph` provides the **add\_node** function to add any node that implements the `Node` trait.

```
let mut graph = Graph::new();
graph.add_node(node);
```

## add edge between nodes

Once the nodes are added to the `Graph`, it is necessary to establish relationships between them by adding edges. The `Graph` provides the **`add_edge`** function to define a directed edge between two nodes, where the source node is connected to one or more target nodes.

For example, after initializing and adding two nodes ( `node` and `node1` ) to the `Graph`, you can establish an edge from `node` to `node1` as follows:

```
let node_id = node.id();
let node1_id = node1.id();
graph.add_edge(node_id, vec![node1_id]);
```

Here, `node.id()` and `node1.id()` retrieve the unique identifiers of the nodes `node` and `node1`, respectively. The `add_edge` function is called with the `node_id` as the source node and a vector containing `node1_id` as the target node(s). This creates a directed edge from `node` to `node1`.

If you need to connect a node to multiple target nodes, you can include all target node IDs in the vector, like this:

```
let target_nodes = vec![node1_id, node2_id, node3_id];
graph.add_edge(node_id, target_nodes);
```

This flexibility allows you to construct complex directed graphs by defining edges between various nodes.

## Execute a Graph and Retrieve Execution Results

After adding nodes and edges to the `Graph`, it is ready to be executed. The `Graph` provides the **`start`** function to begin the execution process, ensuring that all nodes are processed in the correct order based on their dependencies. Once execution is complete, you can retrieve the results produced by individual nodes.

For example, to start a `Graph` and obtain the execution result of a specific node ( `node1` ), you can perform the following steps:

```
graph.start();
let out = graph.execute_states[&node1_id].get_output().unwrap();
```

```
let out: &String = out.get().unwrap();
assert_eq!(out, "Hello world");
```

1. **Start the Graph:** The `start` function begins the execution of the `Graph`, processing all nodes in a manner that respects their dependencies and actions.
2. **Access Execution State:** After execution, the `execute_states` field of the `Graph` contains the state of each node. The state for a specific node can be accessed using its unique identifier ( `node1_id` in this case).
3. **Retrieve the Node Output:**
  - Call `get_output()` on the node's execution state to obtain its output, which is wrapped in an `Option`.
  - Use `unwrap()` to extract the actual output value if it exists.
4. **Validate the Output:** In this example, the output is expected to be of type `String`, so it is further extracted using `get()`. The result can then be validated with assertions, such as checking if the output equals `"Hello world"`.

This process ensures that the `Graph` is executed properly, and the output of individual nodes can be reliably retrieved and verified.

## Parser

More convenient ways to define nodes and build the graph.

## Auto implementation for Node trait

Implementing the `Node` trait for tons of custom types is a bit of a waste of time. for example you need to implement the `id` method for all of them, which just return a `NodeId` but you have to write such code over and over again.

Dagrs provides the [procedural macro](#) `auto_node` to simplify implementing the `Node` trait for custom types. As the following example goes, macro `auto_node` will add neccery fields to your custom type, and implement the `Node` trait for this type.

```
use dagrs::{auto_node}

#[auto_node]
struct MyNode { /*Put customized fields here.*/ }

#[auto_node]
struct MyNodeGeneric<T, 'a> {
    my_field: Vec<T>,
    my_name: &'a str,
}
```

```
#[auto_node]
struct MyUnitNode;
```

Note:

The macro `auto_node` currently doesn't work on [tuple structs](#).

## Auto implementation for build the graph

Dagrs also provides an easy way to build a graph using the `dependencies!` macro. This eliminates the need to manually add nodes and edges one by one, saving time and making the process more straightforward. The macro allows you to define node dependencies directly in a concise and readable format.

Here's an example demonstrating how to use the `dependencies!` macro to construct a graph:

```
use dagrs::dependencies;
let s = MyNode::new(node_name, &mut node_table);
let a = MyNode::new(node_name, &mut node_table);
let b = MyNode::new(node_name, &mut node_table);
let mut g = dependencies!(
    s -> a b,
    b -> a
);
```

The nodes `s`, `a`, and `b` are created using the `MyNode` type, which automatically implements the `Node` trait via the `auto_node` macro. The `dependencies!` macro defines the relationships between nodes in a concise format, where `s -> a b` indicates that there is a channel from `s` to `a` and `b`, and `b -> a` specifies a channel from `b` to `a`. This macro simplifies graph construction by automatically generating the required structure with all specified nodes and their dependencies, avoiding repetitive and boilerplate code.

The `g` variable represents the generated graph, which is automatically constructed by the `dependencies!` macro based on the defined node relationships.

## Conditional Execution

Execute nodes in specific cases.

`ConditionalNode` is a specialized node type used in the Dagrs framework to control the execution flow based on custom conditions. It encapsulates a user-defined condition, represented by the `Condition` trait, which determines whether the workflow should proceed or halt at this node.

It can evaluate the condition using input channels, output channels and environment variables. When the condition is met during execution, Dagr will continue running. If the condition is not met, Dagr will stop on this node.

```
pub struct ConditionalNode {
    id: NodeId,
    name: NodeName,
    condition: Box<dyn Condition>,
    in_channels: InChannels,
    out_channels: OutChannels,
}

impl ConditionalNode {
    /// Creates a new ['ConditionalNode'] with the given name and condition.
    /// The ['Condition'] determines whether execution should continue or stop
    at this node.
    /// The ['NodeTable'] is used to allocate a unique ID for this node.
    pub fn with_condition(
        name: NodeName,
        condition: impl Condition + 'static,
        node_table: &mut NodeTable,
    ) -> Self {
        Self {
            id: node_table.alloc_id_for(&name),
            name,
            condition: Box::new(condition),
            in_channels: InChannels::default(),
            out_channels: OutChannels::default(),
        }
    }
}

#[async_trait]
impl Node for ConditionalNode {
    /// Returns the unique identifier of this conditional node
    fn id(&self) -> NodeId {
        self.id
    }

    /// Returns the name of this conditional node
    fn name(&self) -> NodeName {
        self.name.clone()
    }

    /// Returns a mutable reference to the input channels of this node
    fn input_channels(&mut self) -> &mut InChannels {
```

```

        &mut self.in_channels
    }

    /// Returns a mutable reference to the output channels of this node
    fn output_channels(&mut self) -> &mut OutChannels {
        &mut self.out_channels
    }

    /// Executes the condition logic of this node
    async fn run(&mut self, env: Arc<EnvVar>) -> Output {
        Output::ConditionResult(
            self.condition
                .run(&mut self.in_channels, &self.out_channels, env)
                .await,
        )
    }

    /// Returns true since this is a conditional node
    fn is_condition(&self) -> bool {
        true
    }
}

```

Use cases of the `ConditionalNode` can be found in [example - conditional execution](#).

## Loop Subgraph

Looped nodes in Graph.

LoopSubgraph is a special node type designed to represent a subgraph of nodes that are executed in a loop structure within a larger computational graph. Instead of being executed directly, a LoopSubgraph acts as a container for a set of inner nodes, which are managed and executed repeatedly by the parent graph according to the loop logic.

The LoopSubgraph is included in the main graph as a single node, but internally contains multiple nodes that will be executed repeatedly. The connection and execution of the loop is controlled by the parent graph rather than the LoopSubgraph itself.

```

pub struct LoopSubgraph {
    id: NodeId,
    name: NodeName,
    in_channels: InChannels,
    out_channels: OutChannels,
    // Inner nodes, contains the nodes that need to be executed in a loop
}

```

```
    inner_nodes: Vec<Arc<Mutex<dyn Node>>>,
}
```

LoopSubgraph is implemented by Node trait, so Graph will treat LoopSubgraph as a single node. Users need to add nodes that need to form a loop structure to LoopSubgraph, and then add it to Graph .

Use cases of the LoopSubgraph can be found in [example - loop subgraph](#)

## Send & Recv without id

Send or receive packets without caring about the receiver or sender.

In cases where you want to send or receive packets without caring about the sender or receiver, using send\_to and recv\_from is not a good idea. Therefore, we provide new methods for InChannels and OutChannels to handle this situation.

## Receive packets without sender's id

```
impl InChannels {
    pub async fn recv_any(&mut self) -> Result<(NodeId, Content), RecvErr> {
        let mut futures = Vec::new();
        let ids: Vec<NodeId> = self.keys();

        for id in ids {
            let channel = self.get(&id).ok_or(RecvErr::NoSuchChannel)?;
            let fut = Box::pin(async move {
                let content = channel.lock().await.recv().await?;
                Ok::<_, RecvErr>((id, content))
            });
            futures.push(fut);
        }

        if futures.is_empty() {
            return Err(RecvErr::NoSuchChannel);
        }

        match select_ok(futures).await {
            Ok((result, _)) => Ok(result),
            Err(_) => Err(RecvErr::Closed),
        }
    }
}
```



```
// other methods ...  
}
```

`recv_any` receives data from any available channel and returns both the sender's ID and the content. This method will wait until any channel has data available.

## Send packets without receiver's id

```
impl OutChannels {  
    pub async fn broadcast(&self, content: Content) -> Vec<Result<(),  
    SendErr>> {  
        let futures = self  
            .0  
            .iter()  
            .map(|(c, c)| async { c.lock().await.send(content.clone()).await  
});  
  
        join_all(futures).await  
    }  
  
    // other methods ...  
}
```

Here we can use `broadcast` to send a packet to all the receivers.

Or we can call "get\_receiver\_ids" to get all available receiver node IDs and select one or some of them to send packets.

```
impl OutChannels {  
    /// Returns a list of all available receiver node IDs.  
    pub fn get_receiver_ids(&self) -> Vec<NodeId> {  
        self.0.keys().copied().collect()  
    }  
  
    // other methods ...  
}
```

Use cases of the above methods can be found in [example - receive any & broadcast & typed-action](#).

## typed Action

Limit the input & output type of an Action.

Tired of casting types from Any trait objects? Let's try the TypedAction trait. This trait allows actions to use typed input and output channels instead of raw channel types. Through generic parameters I and O, specific types can be specified for input and output channels, providing compile-time type checking.

```
#[async_trait]
pub trait TypedAction: Send + Sync {
    /// The type parameter for input channels
    type I: Send + Sync + 'static;
    /// The type parameter for output channels
    type O: Send + Sync + 'static;

    /// Converts raw input channels to typed input channels
    ///
    /// # Arguments
    ///
    /// * `in_channels` - The raw input channels
    ///
    /// # Returns
    ///
    /// Returns a typed input channel with the type specified by the
    associated type `I`
    fn make_typed_in_channels(&self, in_channels: &InChannels) ->
TypedInChannels<Self::I> {
        TypedInChannels(in_channels.0.clone(), PhantomData::default())
    }

    /// Converts raw output channels to typed output channels
    ///
    /// # Arguments
    ///
    /// * `out_channels` - The raw output channels
    ///
    /// # Returns
    ///
    /// Returns a typed output channel with the type specified by the
    associated type `O`
    fn make_typed_out_channels(&self, out_channels: &OutChannels) ->
TypedOutChannels<Self::O> {
        TypedOutChannels(out_channels.0.clone(), PhantomData::default())
    }

    /// The method that users need to implement to define their action logic
    ///
    /// This is the main method of the trait, used to execute the specific
```

```

action logic.
    /// It receives typed input and output channels, along with environment
    variables,
    /// and returns the execution result.
    ///
    /// # Arguments
    ///
    /// * `in_channels` - The typed input channels
    /// * `out_channels` - The typed output channels
    /// * `env` - The environment variables
    ///
    /// # Returns
    ///
    /// Returns the result of the action execution
    async fn run(
        &self,
        in_channels: TypedInChannels<Self::I>,
        out_channels: TypedOutChannels<Self::O>,
        env: Arc<EnvVar>,
    ) -> Output;
}

```

Action trait is implemented for TypedAction trait.

```

#[async_trait]
impl<T: TypedAction> Action for T {
    async fn run(
        &self,
        in_channels: &mut InChannels,
        out_channels: &mut OutChannels,
        env: Arc<EnvVar>,
    ) -> Output {
        let in_channels = self.make_typed_in_channels(in_channels);
        let out_channels = self.make_typed_out_channels(out_channels);
        self.run(in_channels, out_channels, env).await
    }
}

```

In most cases you only need to implement the `run` method of `TypedAction`. Use cases can be found in [example - receive any & broadcast & typed-action](#)